コンパイラ (1)

2026/10/02

概要

専門科目 (CS 必修)	2 単位
担当教員	結縁祥治 [ゆうえん しょうじ] 教授
担当 TA	貞本優菜 [さだもと ゆな] (M1)
教科書	補助資料を配布
参考書	コンパイラの理論と作成技法： 大山口通夫、三橋一郎 (サイエンス社)
目的およびねらい	プログラミング言語のコンパイラに関する諸概念と実現法の基礎を修得する。

概要

専門科目 (CS 必修)	2 単位
担当教員	結縁祥治 [ゆうえん しょうじ] 教授
担当 TA	貞本優菜 [さだもと ゆな] (M1)
教科書	補助資料を配布
参考書	コンパイラの理論と作成技法： 大山口通夫、三橋一郎 (サイエンス社)
目的およびねらい	本当は：プログラミング言語、アルゴリズム、 プログラミング演習

講義・演習の進め方 (1)

- 講義については原則対面で実施する. (オンデマンドを３回予定)
- 講義の最後に演習課題を TACT から出題（毎回ではない）
  予定は Web ページで公開 (途中で変更される場合もある)
- TACT に pdf ファイルをアップロードして提出
  手書きレポートをスマホでスキャンする方法は web ページを参照
  pdf 以外で提出された場合は採点できないこともある.
- 演習の提出期限は出題の一週間後の同じ曜日の 23:55 まで
  提出期限後は別途フォームから提出

講義・演習の進め方 (2)

- 講義資料 (講義ページに掲載) に沿って講義を進める。
  必要に応じて参考図書を参照する。
- 評価：定期試験と演習レポート
- 定期試験 (12/1) は対面実施．状況に応じて詳細は検討する．
- 講義ページの URL
  `https://sites.google.com/sqlab.jp/26compiler`
- 質問 Teams のクラスの「質問チャネル」
  参加コードは TACT メッセージで通知

予定

金曜：１限 8:45 – 10:15

火曜：４限 14:45 – 16:15

10/2	序論・字句解析	10/30	SSA 形式
10/6	下向き構文解析	11/6	SSA 上の最適化
10/9	上向き構文解析 (1)	11/10	ループ最適化 [オンデマンド]
10/13	上向き構文解析 (2)	11/13	関数呼び出し・実行時環境 [オンデマンド]
10/16	上向き構文解析 (3)	11/17	レジスタ割当 [オンデマンド]
10/20	意味解析とデータ表現	11/24	SSA のレジスタ割当とφ除去
10/23	３番地コード生成	11/27	目的コード生成
10/27	制御フロー・データフロー解析	12/1	期末試験

進行状況に応じて内容は変更になることもある

コンパイラ

- 抽象度の高いプログラムを抽象度の低いプログラムに変換する処理系
  抽象度の高いプログラムをデジタル計算機で効率よく実行する

compile vt 1. to collect into one work
2. to compose out of materials from other documents

インタープリタ・仮想機械

インタープリタ

- 抽象度の高いプログラムを直接実行する抽象度の低いプログラムで書かれた処理系
  抽象度の高い言語を直接実行する。
  一般に遅い。コンパイラの 100 倍程度。デバッグが簡単

抽象機械

- 抽象度の高いプログラムを分解して、機械語よりも大きい単位で動作させる。
- スタックの操作などを仮定して変換の手間を軽減する
- 再配置可能コードを生成する　（多くの場合メモリ管理のハードで実現）

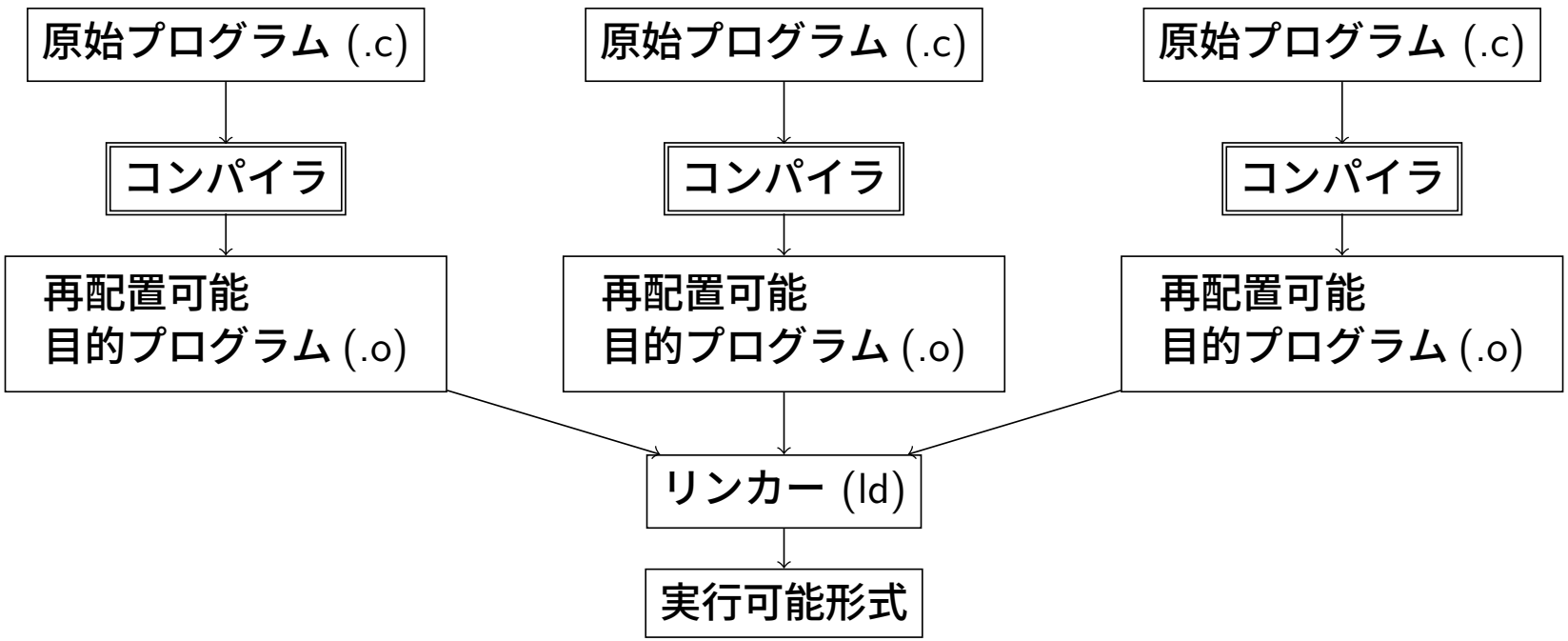
コンパイル

```
#include <stdio.h>
int inc(int x) {
  int y;
  y = x + 1;
  return y;
}
```

`clang -emit-llvm -S inc.ll inc.c` の結果 (一部)

```
define i32 @inc(i32) #0 {
  %2 = alloca i32, align 4
  %3 = alloca i32, align 4
  store i32 %0, i32* %2, align 4
  %4 = load i32, i32* %2, align 4
  %5 = add nsw i32 %4, 1
  store i32 %5, i32* %3, align 4
  %6 = load i32, i32* %3, align 4
  ret i32 %6
}
```

コンパイラの形式



.o: かなり言語の影響をうける (リンカで正当性チェック)

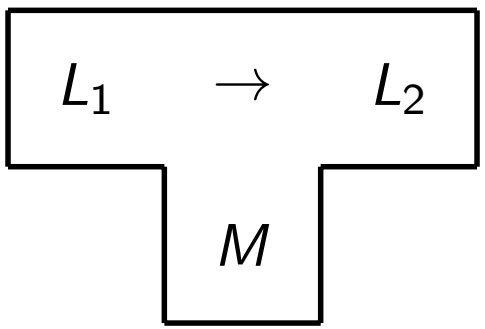
中間言語で出力することもある (.NET など)

構文と意味

- 構文 (Syntax)
  文法を定める規則に基づいたプログラムそのもの
  - プログラムとして正しいかどうかを効率的にチェック
  - 型整合によって意味的制約
- 意味 (Semantics)
  プログラムがどう振る舞うか
  - 入出力関係＝関数
  - 状態遷移機械

コンパイラ：意味を保存しながら、構文間の変換を行う

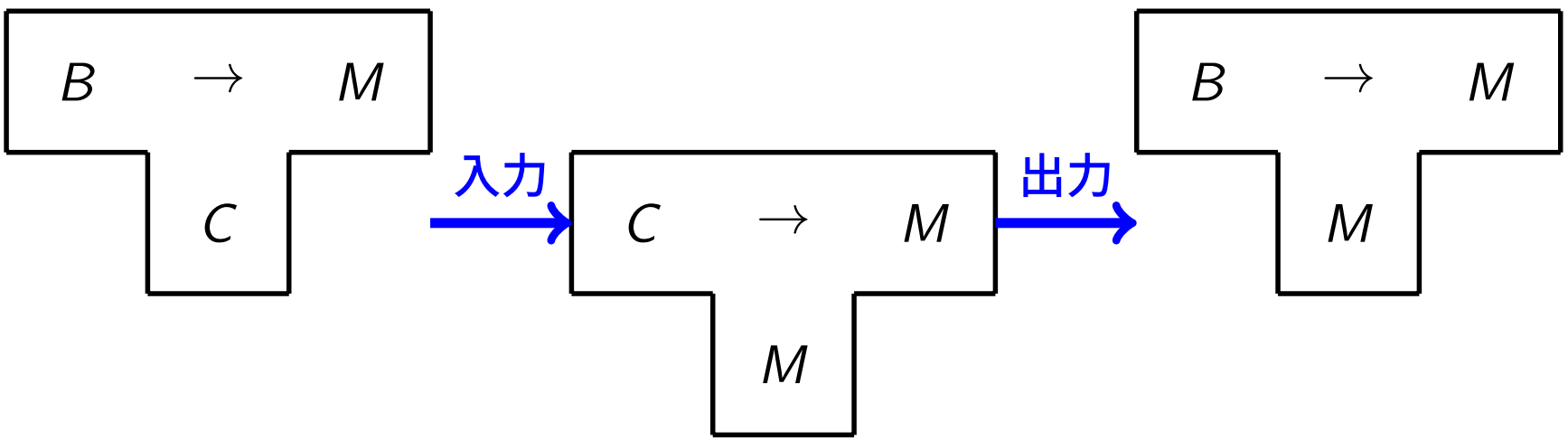
T 図形



原始言語（ソース言語）L_1 を目的言語（ターゲット言語）L_2 に変換する言語処理系が言語 M によって記述されている

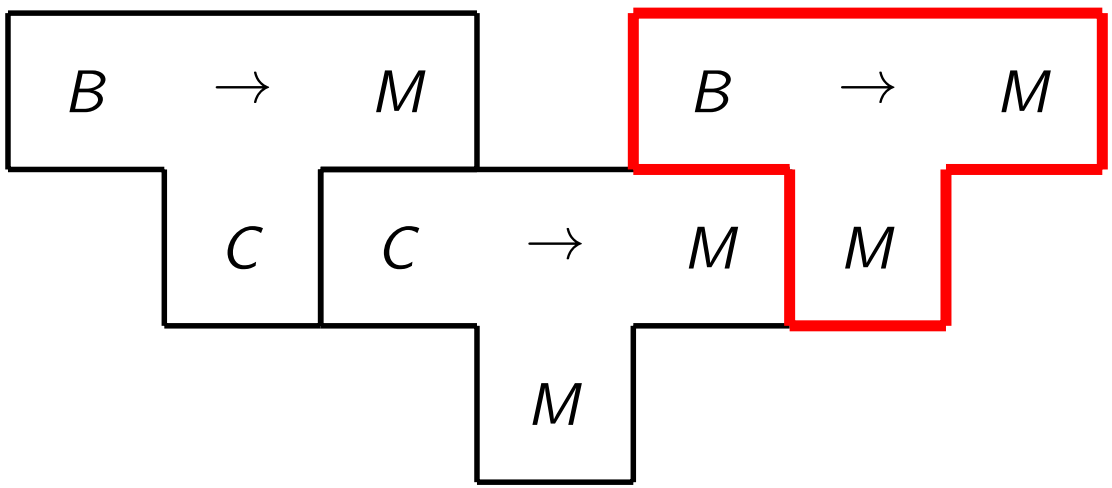
言語処理系

言語 B からマシン語 M へのコンパイラを言語 C で記述し、M 上で動作する C コンパイラを用いて、M で動作する言語 B のコンパイラを作成する。



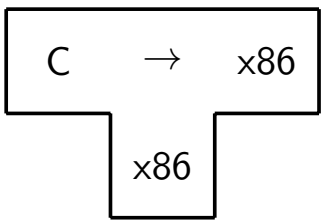
言語処理系

言語 B からマシン語 M へのコンパイラを言語 C で記述し、M 上で動作する C コンパイラを用いて、M で動作する言語 B のコンパイラを作成する。

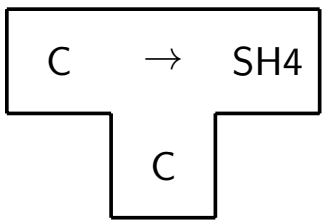


クロスコンパイル/ブートストラップ

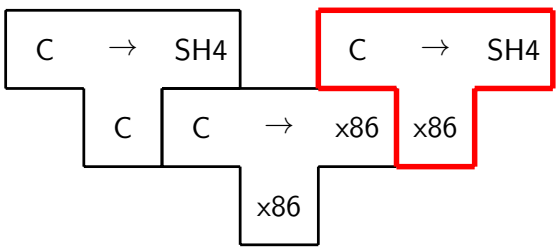
直接開発環境がない計算機のプログラムを作成する



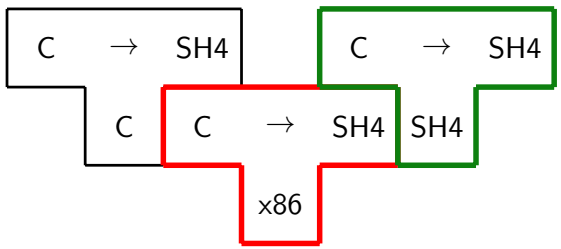
x86 の C コンパイラ



C で記述した SH4 の C コンパイラ

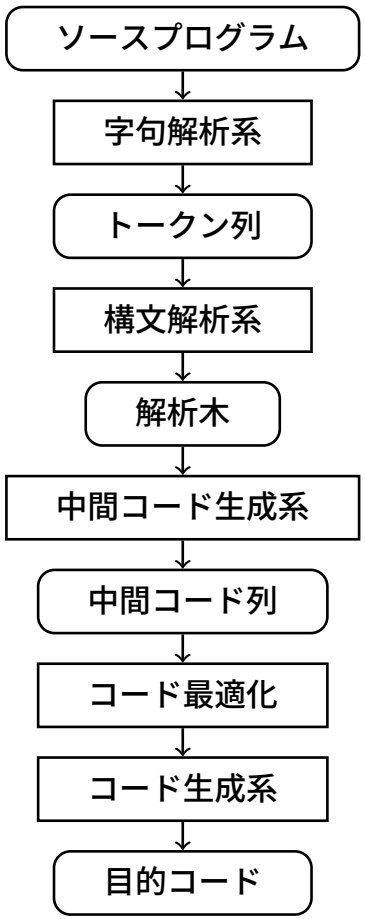


x86 による SH4 のクロスコンパイラ



SH4 の C コンパイラ

コンパイラの基本構成



- 字句解析
  ソースプログラムから単語を切りだす
  オートマトン
- 構文解析
  文法に従って、単語間の関係として木構造を生成
  再帰降下構文解析、LR 構文解析
- 中間コード生成
  木構造が示す意味に従って中間的な実行コードを作成
  3 番地コード、記号表
- コード最適化
  意味を変えずによりより実行が高速なコードに変換
  DAG, データフロー解析、SSA
- コード生成系
  実行アーキテクチャに適した高速な目的コードを生成
  レジスタ割り当て

字句解析

字句解析の役割

```
main(int argc, char **argv) {
  printf("Hello World!\n");
}
```

```
% hexdump -b hello.c
0000000 155 141 151 156 050 151 156 164 040 141 162 147 143 054 143 150
0000010 141 162 040 052 052 141 162 147 166 051 040 173 012 011 160 162
0000020 151 156 164 146 050 042 110 145 154 154 157 040 127 157 162 154
0000030 144 041 134 156 042 051 073 012 175 012
000003a
```

```
(Id,"main")(Del,"(")(KW,"int")(Id,"argc")(Del,",")
(KW,"char")(Op,"*")(Op,"*")(Id,"argv")(Del,")")
(Del,"{")(Id,"printf")(Del,"(")
(String,"Hello World!\n")(Del,")")(Del,";")(Del,"}")
```

ソースプログラムから字句の属性と値の対の系列を作成する

字句解析の手法

プログラミング言語の単語（正規言語）に対するオートマトンを作成

正規言語 →NFA→DFA→mDFA(状態最小化)

構成するオートマトン (mDFA)

DFA の受理状態に属性と値を記憶する

効率をあげるために状態を最小化する

オートマトンが受理状態 ⇒ 受理状態の属性と値を出力

値 · 属性：外部変数で実現 (オートマトン自体とは別)

正規表現

アルファベット：記号の (有限) 集合
The (English) Alphabet:$\{A, B, C, \cdots, Z\}$

定義 3.1 アルファベット Σ 上の正規表現

(1) ε は正規表現

(2) すべての $a \in \Sigma$ は正規表現

(3) R_A,R_B が正規表現であるとき、
$R_A \mid R_B$, $(R_A)(R_B)$, $(R_A)^*$ は正規表現

$R_A|R_B$:R_A と R_B の選択
$R_A =''ab''$,$R_B =''cd''$ のとき $R_A|R_B =''ab''$または$''cd''$

$(R_A)(R_B)$:R_A と R_B の連接
$R_A =''ab''$,$R_B =''cd''$ のとき $(R_A)(R_B) =''abcd''$

$(R_A)^*$:R_A の 0 回以上の繰り返し
$R_A =''ab''$ のとき $(R_A)* = \varepsilon$または$''ab''$または$''abab''$または$''ababab'' \cdots$

有限オートマトン

定義 3.3' 有限オートマトン (FA) $M = \langle Q, \Sigma, \delta, q_0, F \rangle$

Q	状態の有限集合
Σ	(入力) アルファベット ($\varepsilon \notin \Sigma$)
$\delta \subseteq Q \times (\Sigma \cup \{\varepsilon\}) \times Q$	状態遷移
$q_0 \in Q$	初期状態
$F \subseteq Q$	受理状態集合

受理言語 $L(M) = \{w \in \Sigma^* \mid (q_0, w, q_f) \in \delta^*, q_f \in F\}$

$\delta^* = \bigcup_{0 \leq i \leq n} \delta^n$

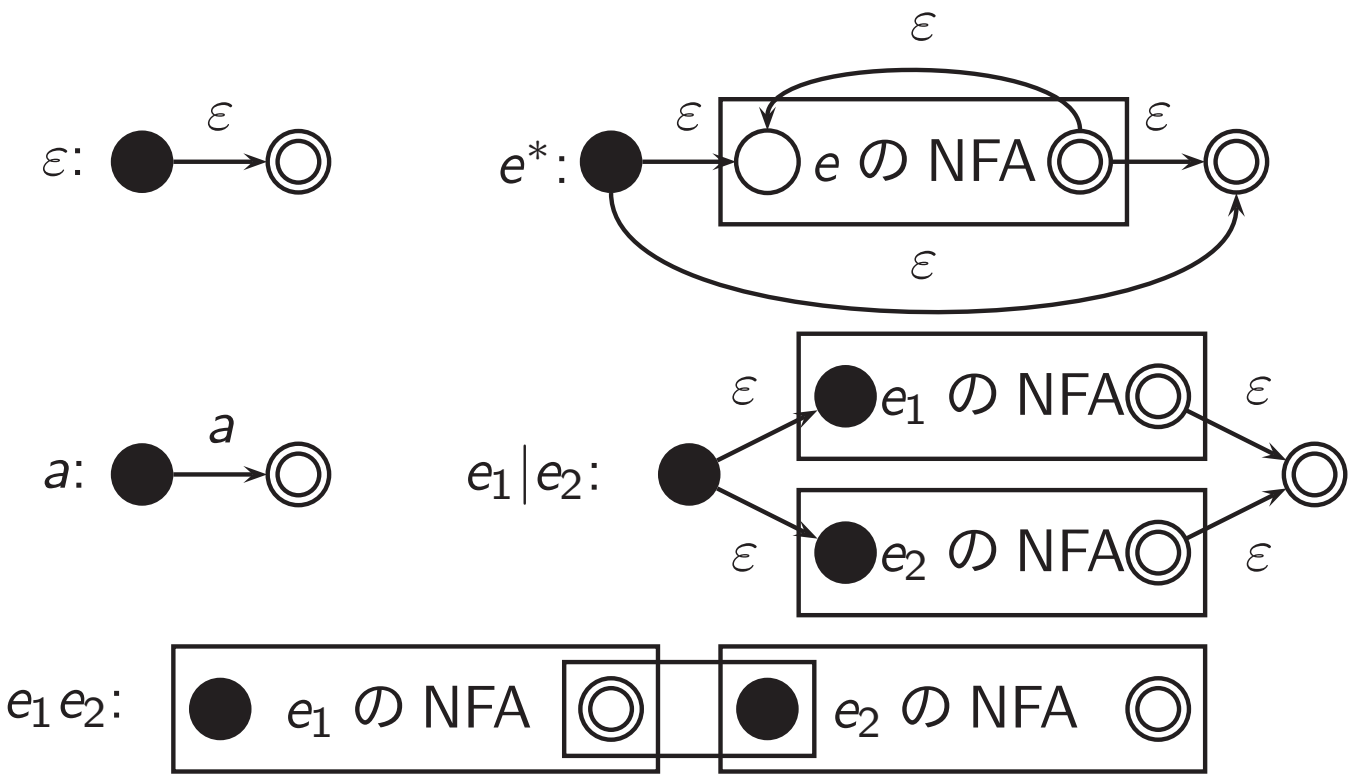
$\delta^0 = \{(q, \varepsilon, q) | q \in Q\}$

$\delta^{k+1} = \{(q, wa, q') | \exists q''.(q, w, q'') \in \delta^k \ \wedge \ (q'', a, q') \in \delta\}$

δ の遷移に空文字 ε を含まず、δ が決定的 ($\{q' | \delta(q, a, q'), a \in \Sigma\}$ が単一集合) のとき、決定性有限オートマトン (DFA)

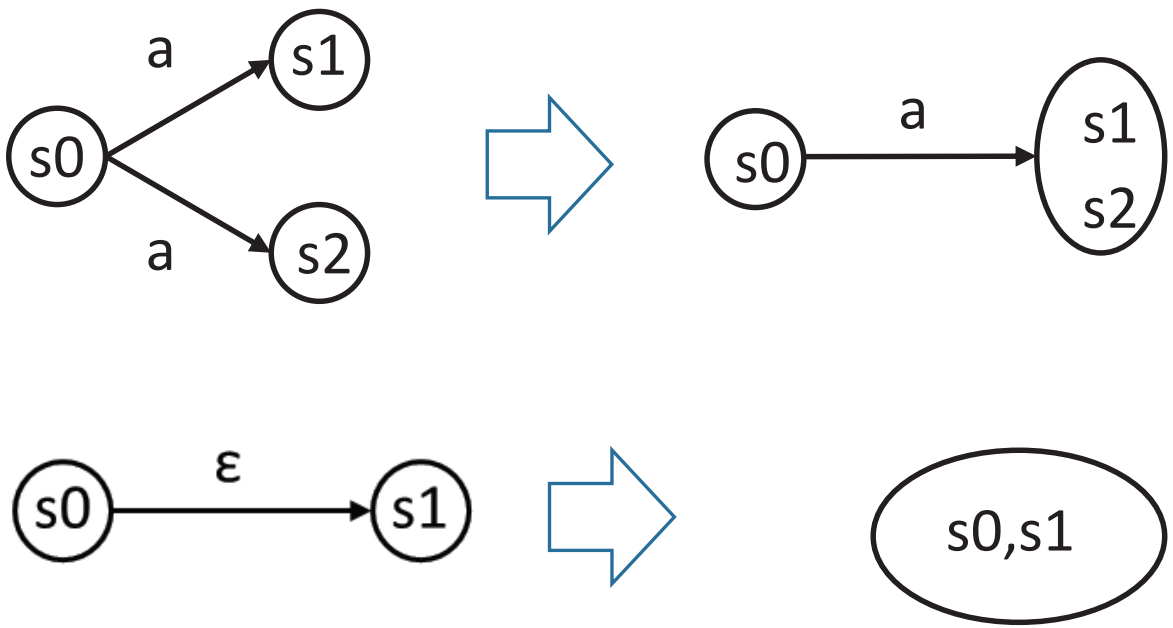
正規表現⇒NFA

非決定性=ε 遷移

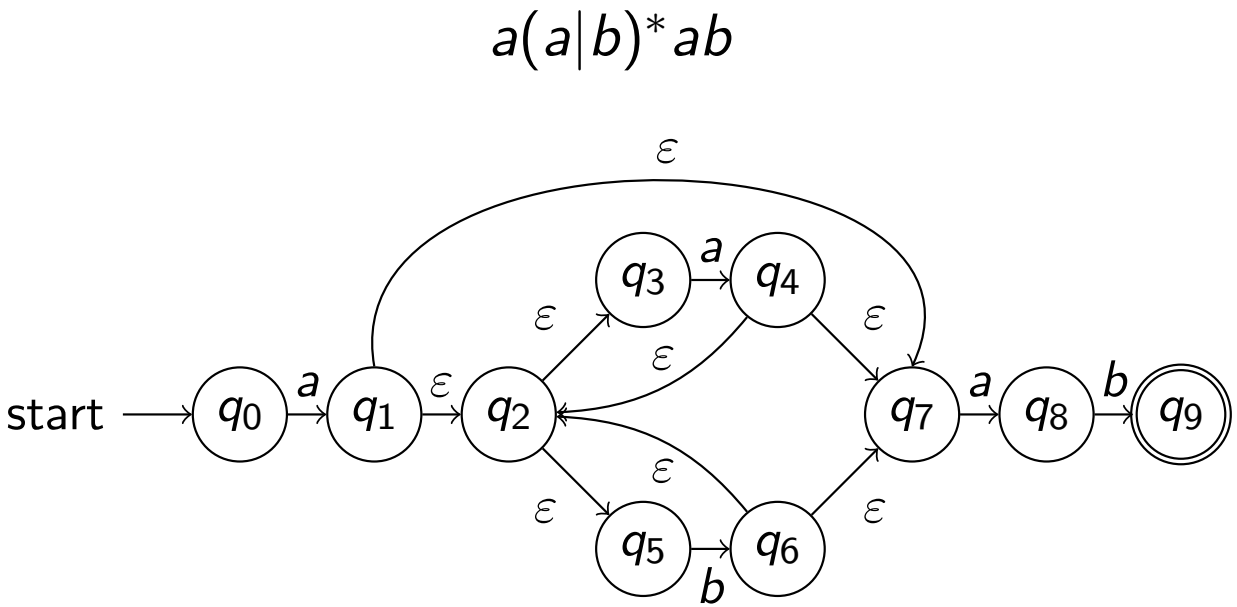


決定性オートマトンへの変換

- **部分集合構成法** (subset construction)
  - $(Q, \Sigma, \delta, q_0, F)$ から $(Q_D, \Sigma, \delta_D, \mathrm{cl}(q_0), F_D)$ を構成
  - $\mathrm{cl}(S) = \{q \mid s \xrightarrow{\varepsilon}{}^{*} q, s \in S\}$,
    $\mathrm{goto}(S, a) = \{q' \mid q \xrightarrow{a} q'' \wedge q'' \xrightarrow{\varepsilon}{}^{*} q' \wedge q \in S\}$



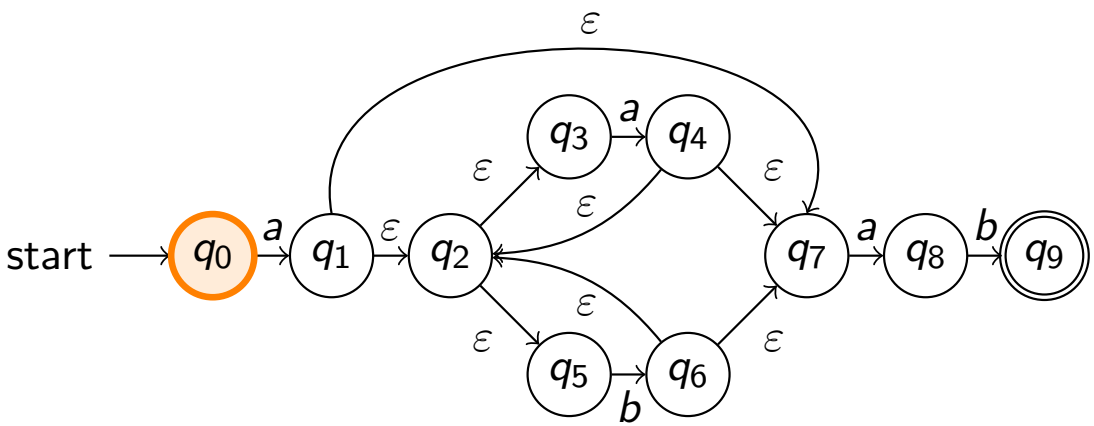
正規表現から構成される NFA



連接、選択、反復を状態と遷移で表す。

DFA への変換

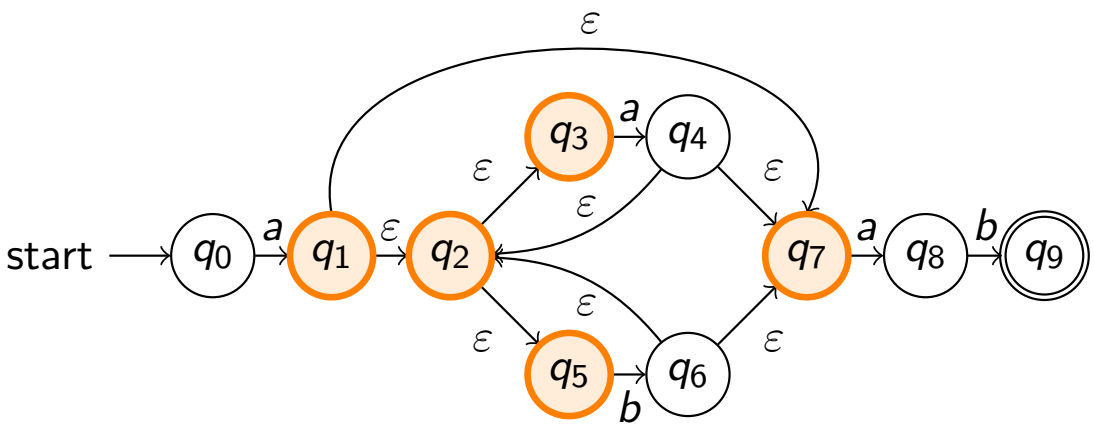
$$D_0 = \mathrm{cl}(\{q_0\}) = \{q_0\}$$



初期状態から文字を読まずに到達できる状態を集める。

DFA への変換

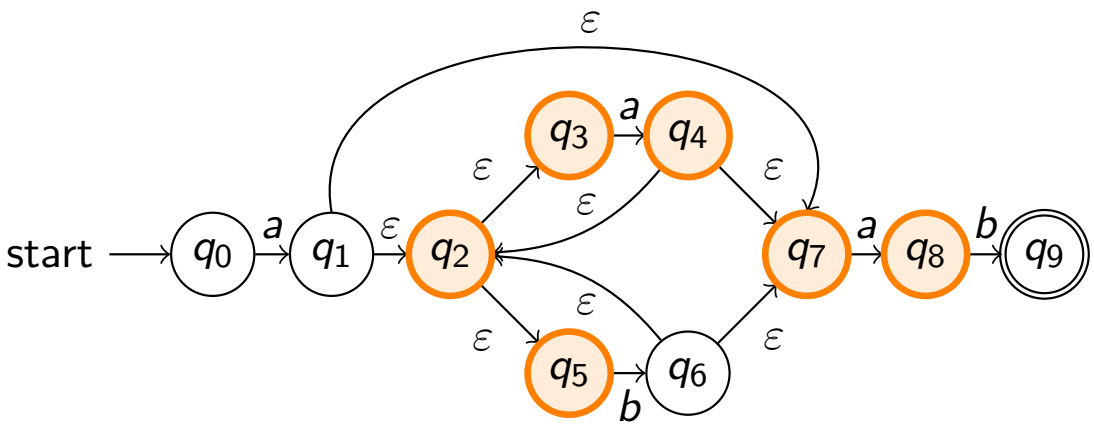
$$\mathrm{goto}(D_0, a) = \{q_1\} \quad D_1 = \mathrm{cl}(\{q_1\}) = \{q_1, q_2, q_3, q_5, q_7\}$$



a を読んだ後、ε 遷移で到達できる状態を集める。

DFA への変換

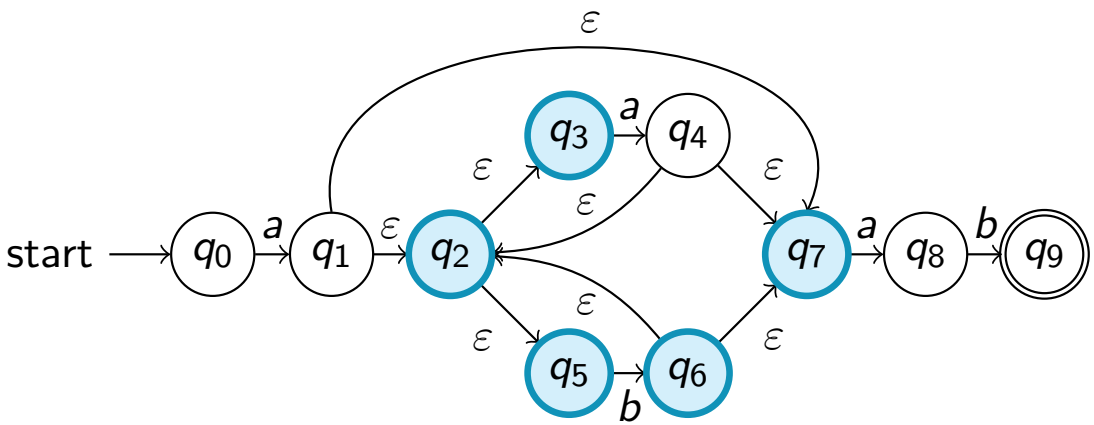
$$D_1 \xrightarrow{a} \{q_4, q_8\} \quad D_2 = \{q_2, q_3, q_4, q_5, q_7, q_8\}$$



q_4 から反復部へ戻り、q_8 は末尾の b を待つ。

DFA への変換

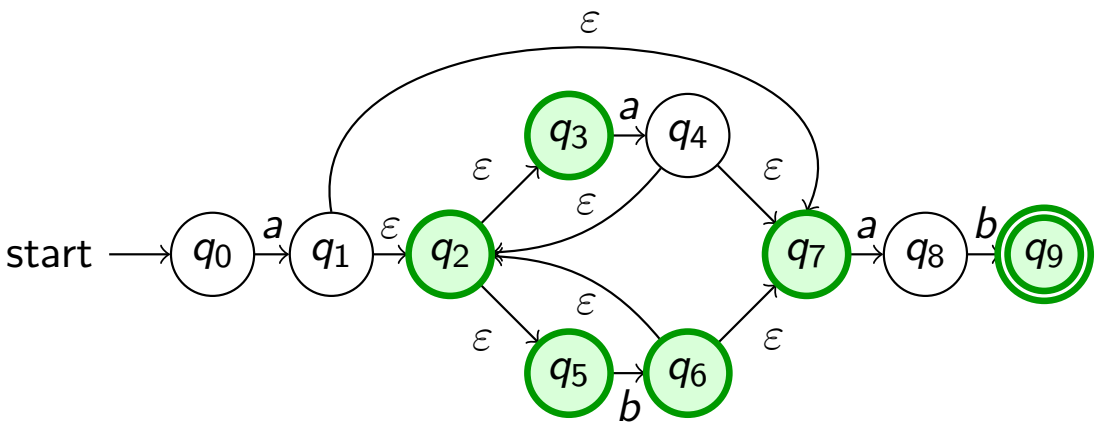
$$D_1 \xrightarrow{b} \{q_6\} \quad D_3 = \{q_2, q_3, q_5, q_6, q_7\}$$



q_6 の閉包を取ると、反復部の入口と脱出口へ広がる。

DFA への変換

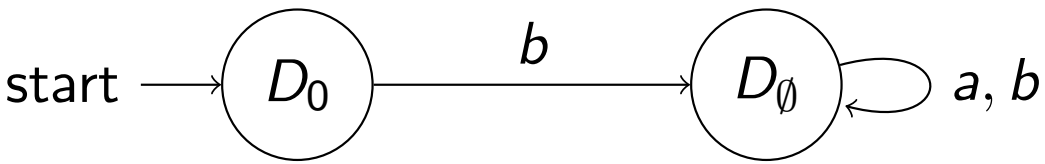
$$D_2 \xrightarrow{b} \{q_6, q_9\} \quad D_4 = \{q_2, q_3, q_5, q_6, q_7, q_9\}$$



q_9 を含むため、D_4 は DFA の受理状態になる。

DFA への変換

$$D_0 \xrightarrow{b} \emptyset \qquad D_\emptyset = \emptyset$$



空集合は、どの入力でも空集合へ戻る死状態になる。

DFA への変換

DFA 状態	対応する NFA 状態	a	b
D_0	$\{q_0\}$	D_1	$D_\emptyset$
D_1	$\{q_1, q_2, q_3, q_5, q_7\}$	D_2	D_3
D_2	$\{q_2, q_3, q_4, q_5, q_7, q_8\}$	D_2	D_4

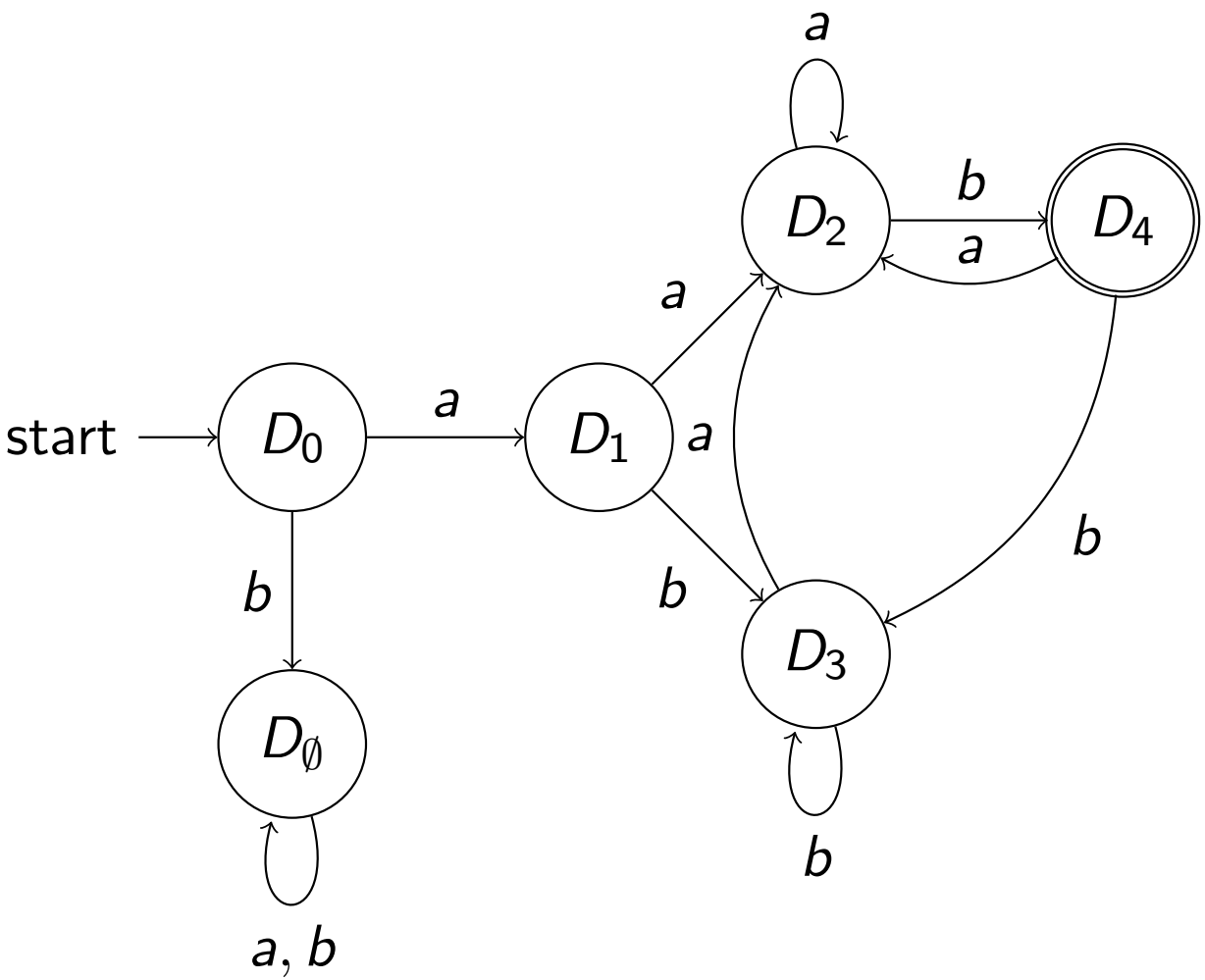
未処理の状態集合を順に取り出し、a, b に対する遷移を計算する。

DFA への変換

DFA 状態	対応する NFA 状態	a	b
D_0	$\{q_0\}$	D_1	$D_\emptyset$
D_1	$\{q_1, q_2, q_3, q_5, q_7\}$	D_2	D_3
D_2	$\{q_2, q_3, q_4, q_5, q_7, q_8\}$	D_2	D_4
D_3	$\{q_2, q_3, q_5, q_6, q_7\}$	D_2	D_3
D_4	$\{q_2, q_3, q_5, q_6, q_7, q_9\}$	D_2	D_3
$D_\emptyset$	$\emptyset$	$D_\emptyset$	$D_\emptyset$

既出の集合と一致した場合は、その DFA 状態への遷移として記録する。

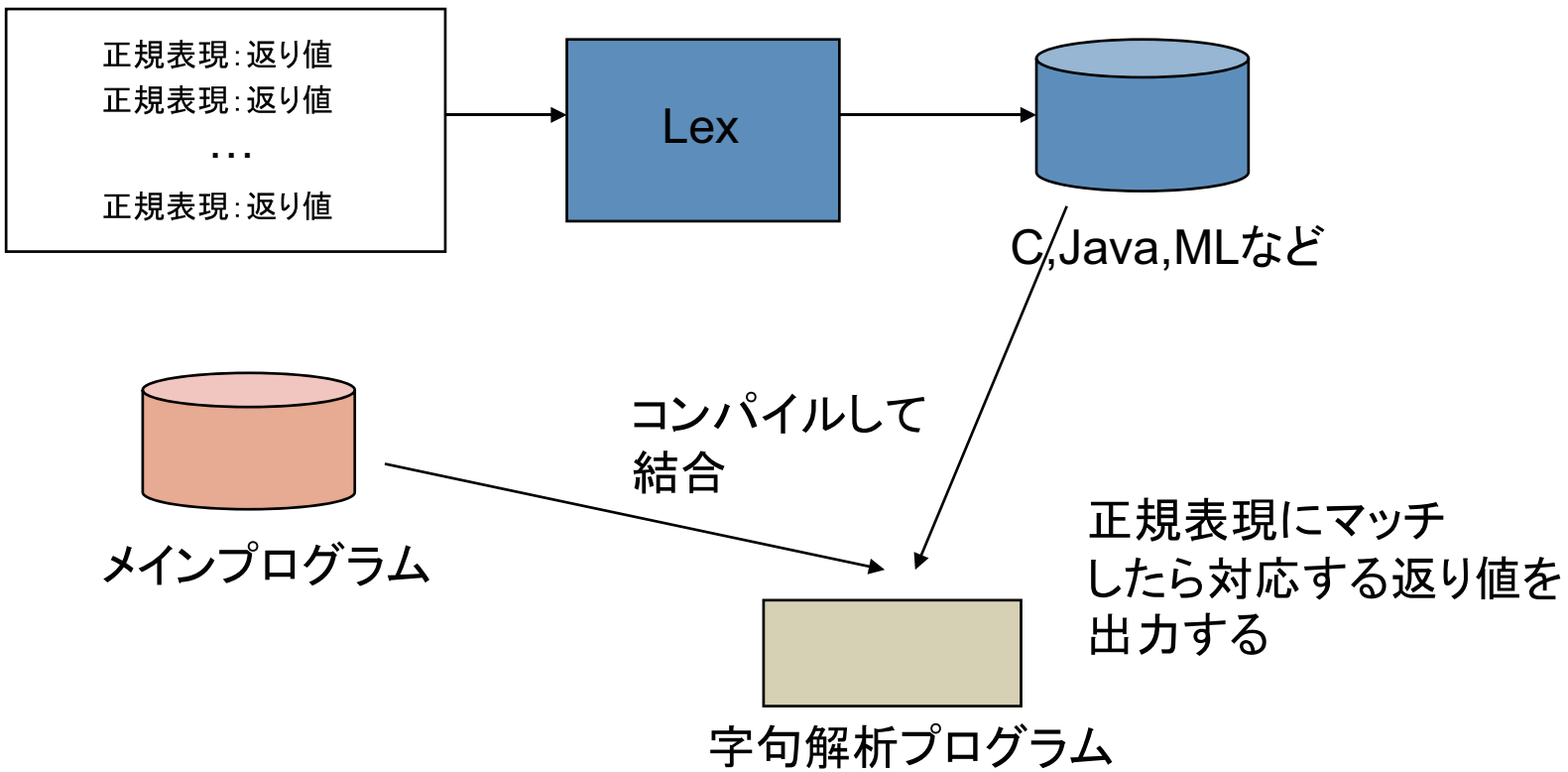
DFA への変換



$q_9 \in D_4$ なので、D_4 を受理状態とする。

Lex の利用

Lex: lexical analizer generator
GNU 版：flex



Lex プログラム

lex1.l

```
%% 正規表現        対応する動作
[A-Za-z]+   {printf("String found:%s\n",yytext);}
[0-9]+      {printf("Digits found:%s\n",yytext);}
.           {printf("Other char found:%d\n",yytext[0]);}
%%
```

実行方法

```
lex␣-o␣lex1.c␣lex1.l      (学科計算機 (ICE))
flex␣-o␣lex1.c␣lex1.l     (それ以外)
cc␣-o␣lex1␣lex1.c␣-ll
```

lex1.c という字句解析プログラムを正規表現と対応動作さから生成

1. 正規表現を受理する DFA を生成
2. 受理状態に達したとき動作を実行する

SLY(Python Lex Yacc)

```
from sly import Lexer

class SimpleLexer(Lexer):
    # Set of token names.   This is always required
    tokens = {  トークンの集合 }

    # 無視する文字
    ignore = ' \t'

    # Regular expression rules for tokens
    トークン = r'正規表現'
      .....

if __name__ == '__main__':
    data = 'x = 3 + 42 * (s - t)'
    lexer = SimpleLexer()
    for tok in lexer.tokenize(data):
        print('type=%r, value=%r' % (tok.type, tok.value))
```

tokens にトークンの名前を列挙して，下のその名前のトークンを正規表現で定義

python プログラムを生成するわけでなく，このプログラムで字句解析